

batch

מערכות הפעלה

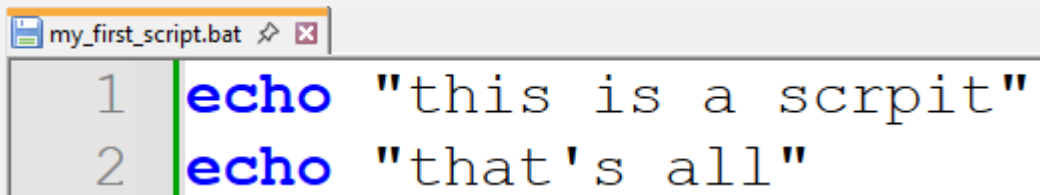
אינטרפטציה לעומת קומפילציה

- בהינתן תוכנית, יש שני סוגי תהליכים להרצה שלו
- **קומפילציה:** קריאת כל הקוד והמרה לקובץ תוכנה המתאים למערכת מסויימת ← הרצת התוכנה
- מבוצע ע"י תוכנה הנקראת קומפיילר (compiler)
- שפות המשתמשות בקומפילציה נקראות compiled languages
- דוגמאות לשפות המשתמשות בקומפילציה: Java, C, C#
- **אינטרפטציה:** קריאה והרצה של כל פקודה בקוד
- מבוצע ע"י תוכנה הנקראת אינטרפרטר (interpreter)
- שפות המשתמשות באינטרפטציה נקראות interpreted languages
- דוגמאות לשפות המשתמשות באינטרפטציה: Python, JavaScript
- הערה: רוב השפות המודרניות משלבות אלמנטים מסויימים מהתהליך האחר
- Interpreted languages מכונות גם שפות סקריפטינג (scripting) ותוכניות שלהן מכונות גם סקריפט

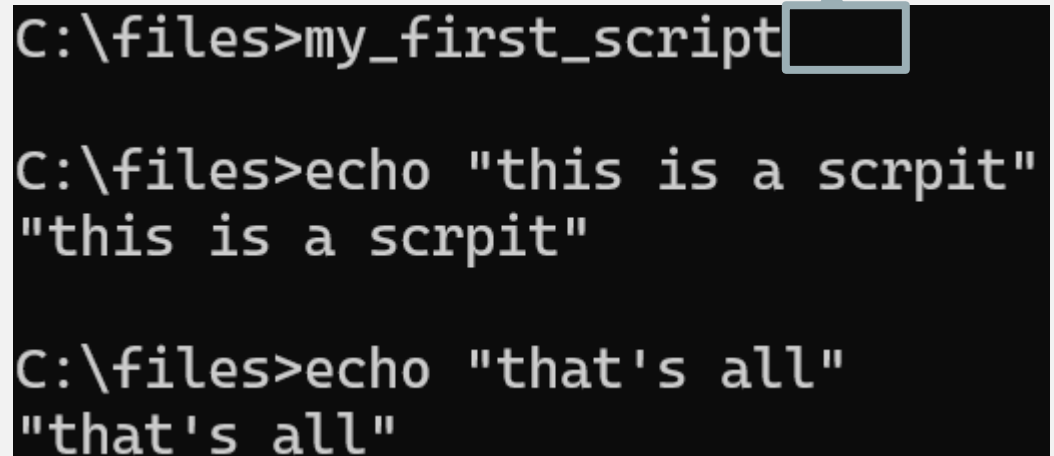
BATCH

- Batch (באנגלית: קבוצה) היא שפת סקריפטיניג עבור מערכות הפעלה Windows (וגם DOS)
- היא בעצם הרחבה של השפה של CMD – ניתן להשתמש בה בכל פקודת CMD, בנוסף לפקודות ומבנים אחרים
- הסיומת bat משמשת לייצג קבצי סקריפט של batch
- ניתן להריץ אותם בCMD כמו תוכנה, כולל העובדה שניתן להשמיט את הסיומת
- דוגמא

אין צורך להוסיף את הסיומת



```
1 echo "this is a scrpit"
2 echo "that's all"
```



```
C:\files>my_first_script

C:\files>echo "this is a scrpit"
"this is a scrpit"

C:\files>echo "that's all"
"that's all"
```

BATCH

- ניתן גם להריץ script ע"י הפקודה call ואחריה כתובת הסקריפט
- באופן זה ניתן להריץ סקריפטים שנמצאים בתיקיות אחרות

```
C:\files>cd dir
```

```
C:\files\dir>call ../my_first_script  
"this is a scrpit"  
"that's all"
```

```
my_first_script.bat
1 echo "this is a scrpit"
2 echo "that's all"
```

BATCH

- אנו יכולים לראות שהרצה של סקריפט שקולה לכתיבת שורות הסקריפט פקודה אחר פקודה
- ההצגה המפורשת של כל פקודה מייצרת הצגה מסורבלת ופחות נוחה
- ניתן להוסיף @ לפני הפקודה כדי שלא תוצג ההזנה שלה
- כדי שזה יקרה באופן גורף, נוכל לשנות את הסטטוס של echo ל off

```
C:\files>my_first_script
C:\files>echo "this is a scrpit"
"this is a scrpit"
C:\files>echo "that's all"
"that's all"
```

```
@echo "this is a scrpit"
echo "that's all"
```

נמנע הצגה
של הפקודה
שמשנה את
הסטטוס

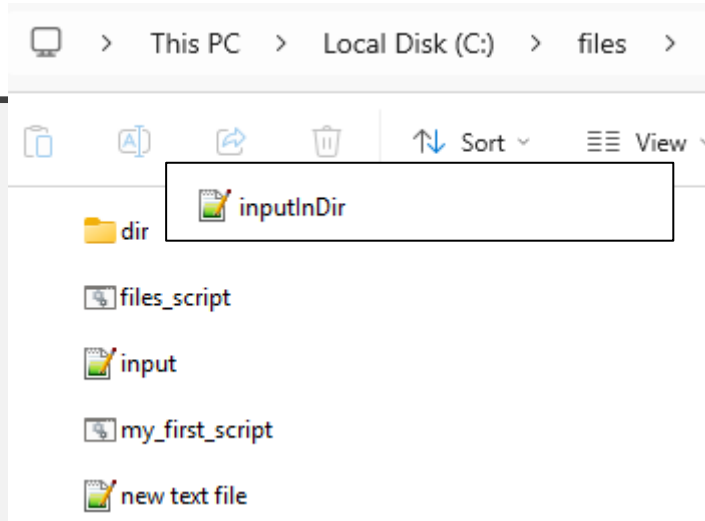
```
C:\files>my_first_script
"this is a scrpit"
C:\files>echo "that's all"
"that's all"
```

```
@echo off
echo "this is a scrpit"
echo "that's all"
echo on
```

נחזיר לסטטוס on, אחרת יישאר off לאחר ריצה הסקריפט

```
C:\files>my_first_script
"this is a scrpit"
"that's all"
```

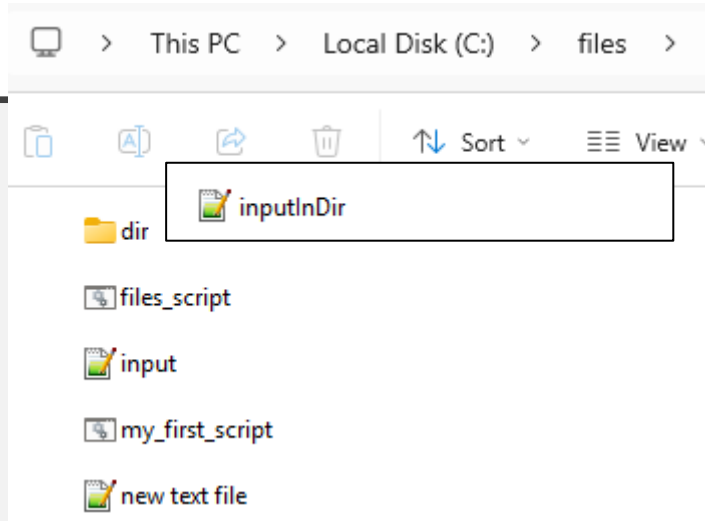
תרגול



- כתבו סקריפט אשר אשר מייצר קובץ אשר מכיל:
הודעה שאומרת "רשימת קבצים:"
- רשימה של הקבצים (בלבד, ללא תת-תיקיות) בתיקייה ובתת התיקיות שלהם בתצורה פשוטה (השמות בלבד)
- בסוף הסקריפט תודפס הודעה שאומרת "הודפס ל>שם הקובץ"<
- דוגמא:

```
C:\files>files_script.bat
printed to "list_of_files.txt"
C:\files>type list_of_files.txt
list of files:
C:\files\files_script.bat
C:\files\input.txt
C:\files\list_of_files.txt
C:\files\my_first_script.bat
C:\files\new text file.txt
C:\files\dir\inputInDir.txt
```

תרגול



- כתבו סקריפט אשר אשר מייצר קובץ אשר מכיל:

- הודעה שאומרת "רשימת קבצים:"

- רשימה של הקבצים (בלבד, ללא תת-תיקיות) בתיקייה ובתת התיקיות שלהם בתצורה פשוטה (השמות בלבד)

- בסוף הסקריפט תודפס הודעה שאומרת "הודפס ל>שם הקובץ">

- דוגמא:

```
@echo off
echo list of files: > list_of_files.txt
dir /b /s /a-d >> list_of_files.txt
echo printed to "list_of_files.txt"
echo on
```

```
C:\files>files_script.bat
printed to "list_of_files.txt"
C:\files>type list_of_files.txt
list of files:
C:\files\files_script.bat
C:\files\input.txt
C:\files\list_of_files.txt
C:\files\my_first_script.bat
C:\files\new text file.txt
C:\files\dir\inputInDir.txt
```

הערות

- כדי להוסיף הערה בסקריפט ניתן להוסיף rem או :: בתחילת השורה
- כדי להוסיף הערה בסוף השורה יש להוסיף & לפני

```
@echo off
echo this is shown
rem echo this is not shown
:: echo neither is this
echo not a comment: rem comment_end
echo is a comment: & rem comment_end
echo not a comment: :: comment_end
echo is a comment: & :: comment_end
echo on
```

```
C:\files>comments
this is shown
not a comment: rem comment_end
is a comment:
not a comment: :: comment_end
is a comment:
```

משתנים

- הפקודה set מאפשרת הגדרה של משתנים

```
set <var_name>=<string>
```

- מבנה הפקודה:

- <var_name> שם המשתנה

- <string> ערך למשתנה

- ניתן לשנות ערך משתנה לאחר שהוא נקבע

- כדי לגשת למשתנה נשים אותו בין שני %

- כדי לכתוב את הסימן % נכתוב %%

```
@echo off
set my_var=first value
echo %my_var%
set my_var=second value
echo %my_var%
echo on
```

```
C:\files>vars
first value
second value
```

```
@echo off
set my_var=first value
echo %%my_var%%=%my_var%
set my_var=second value
echo %%my_var%%=%my_var%
echo on
```

```
C:\files>vars
%my_var%=first value
%my_var%=second value
```

משתנים

```
@echo off
set var_space =hi1
echo %var_space%
echo on
```

```
C:\files>vars_space
ECHO is off.
```

```
:: @echo off
set var_space =hi1
echo %var_space%
:: echo on
```

```
C:\files>set var_space =hi1
```

```
C:\files>echo
ECHO is on.
```

```
@echo off
echo %not_a_var%
echo on
```

```
C:\files>vars_space
ECHO is off.
```

```
@echo off
set var_space= hi1
echo %var_space%
echo on
```

```
C:\files>vars_space
hi1
```

- הפקודה set מאפשרת הגדרה של משתנים
- שימו לב שאין להוסיף רווח לפני הסימן =
- מה קרה פה?
- נשאר את echo בסטטוס on כדי לבדוק
- אין במתשנה כלום לכן קיבלנו echo בלי כלום – הצגת הסטטוס
- אותו דבר יקרה עם משתנה שלא הוגדר
- ניתן להוסיף רווח אחרי ה=, אבל הוא יהיה חלק מהערך

משתנים

```
::var_nums.bat
@echo off
set /a num1=5
set /a num2=10+5
set /a num3=(11-10)*2
set /a num4=(%num2%/ %num1%) / %num3%
echo %num1%
echo %num2%
echo %num3%
echo %num4%
echo on
```

```
C:\files>var_nums
5
15
2
1
```

```
::var_input.bat
```

```
@echo off
set /p name=What's you name?
echo Hello %name%
echo on
```

```
C:\files>var_input
What's you name? Avi
Hello Avi
```

- הפקודה set מאפשרת הגדרה של משתנים
- אפשרויות:
 - ניתן להוסיף /a כדי לשמור ערך מספרי (שלם)
 - ניתן להוסיף % כדי לקבל ערך מהמשתמש
- במקרה זה הערך שאנו מזינים (אחרי =) הוא לא הערך שיכנס למשתנה, אלא הודעה שתצא למשתמש

משתנים

- שימו לב שההגדרה של המשנה היא גלובלית – כלומר היא נשארת גם אחרי שהסקריפט הסתיים
- הפקודה setlocal גורמת לכך שכל משתנה שמוגדר אחריה יוגדר לוקלית
- הפקודה endlocal מוחקת את כל המשתנים הלוקלים (כלומר אלה שהוגדרו אחרי setlocal)
- שימו לב שמשתנה לוקלי ימחק בסוף הסקריפט, גם אם אין endlocal
- הכרחי כאשר נרצה להגדיר משתנה גלובלי לאחר משתנים לוקלים
- כתיבה נכונה יותר גם כשלא

```
::var_create.bat
@echo off
set global_var=val1
setlocal
set local_var=val2
endlocal
echo on
```

```
::var_use.bat
@echo off
echo %%global_var%%=%global_var%
echo %%local_var%%=%local_var%
echo on
```

```
C:\files>var_create
C:\files>var_use
%global_var%=val1
%local_var%=
```

משתנים

- הגדרה של משתנה בעצם מגדירה משתנה סביבה זמני אשר קיים רק בsession הנוכחי של cmd
- ניתן גם לגשת למשתני הסביבה הקבועים

```
::var_env.bat
@echo off
echo os is %os%
echo there are %NUMBER_OF_PROCESSORS% processeors
set /a plus_one = %NUMBER_OF_PROCESSORS% + 1
echo if we had one more there would be %plus_one%
echo on
```

```
C:\files>var_env
os is Windows_NT
there are 8 processeors
if we had one more there would be 9
```

משתנים

- כדי להציג באיזו תיקיה אנחנו ניתן להשתמש במשתנה cd (current directory)

```
::var_env.bat  
@echo off  
echo we're in %cd%  
echo on
```

```
C:\files>var_dir  
we're in C:\files
```

- שימו לב שהערך של cd נקבע לפי התיקיה בה אנו נמצאים ולא התיקיה בה הסקריפט נמצא

```
C:\files>cd dir  
  
C:\files\dir>call ../var_dir  
we're in C:\files\dir
```

תנאים

```
::var_if.bat
@echo off
set /p x=x=
set /p y=y=
set /a sum=x+y
echo x+y=%sum%
if %sum% GTR 0 echo positive
set /a mod=%sum% %% 2
if %mod% == 0 (echo even) else (echo odd)
echo on
```

```
C:\files>var_if
x=4
y=6
x+y=10
positive
even
```

```
C:\files>var_if
x=4
y=-7
x+y=-3
odd
```

- ניתן להשתמש בתנאים
- פעולות

Batch	JAVA
EQU או ==	==
NEQ	!=
LSS	<
LEQ	<=
GTR	>
GEQ	>=
AND	&&
OR	
NOT	!

תרגול

- כתבו סקריפט אשר מקבל מהמשתמש שני מספרים ומדפיס מי המקסימלי
- דוגמאות

```
C:\files>ex_if
x=4
y=6
maximal value is
6
C:\files>ex_if
x=4
y=-7
maximal value is
4
```

תרגול

- כתבו סקריפט אשר מקבל מהמשתמש שני מספרים ומדפיס מי המקסימלי
- דוגמאות

```
::ex_if.bat
@echo off
set /p x=x=
set /p y=y=
echo maximal value is
if %x% GTR %y% (echo %x%) else (echo %y%)
echo on
```

```
C:\files>ex_if
x=4
y=6
maximal value is
6
C:\files>ex_if
x=4
y=-7
maximal value is
4
```

לולאות

- מבנה for מאפשר לנו להגדיר לולאה

```
for <var_name> in <set> do <command>
```

- <var_name> - שם המשתנה, מתחיל ב%% ומורכב מאות אחת בלבד
- <set> אוסף הערכים עליו הלולאה תעבור, נמצא בתוך סוגריים ובתוכו ביטוי המייצג אוסף ערכים
- הפשוט ביותר הוא רצף ערכים עם רווחים, נראה עוד אפשרויות בהמשך
- <command> הפקודה לביצוע

- דוגמא:

```
@echo off  
for %%s in (show me line by line) do echo %%s  
echo on
```

```
C:\files>script.bat  
show  
me  
line  
by  
line
```

לולאות

- שימו לב שירידת שורה מסמלת סוף פקודה
- אם ננסה לרדת שורה כדי לכתוב את קוד הלולאה, זו תהיה כבר פקודה חדשה

```
@echo off
for %%s in (show me line by line) do
echo %%s
echo on
```

```
C:\files>script.bat
The syntax of the command is incorrect.
```

- שתי פתרונות:
- פתרון כללי: הוספת התו ^ לפני ירידת שורה יגרום לכך שירידת השורה לא תיחשב כסוף פקודה

```
@echo off
for %%s in (show me line by line) do ^
echo %%s
echo on
```

```
C:\files>script.bat
show
me
line
by
line
```

- זה נכון באופן כללי, לא רק בלולאות

```
echo show me as ^
one line
```

```
C:\files>script.bat
show me as one line
```

לולאות

- פתרון ספציפי ללולאות: ניתן לפתוח סוגריים (רגילים) אחרי do

```
for %%s in (show me line by line) do (  
    echo %%s  
)
```

```
C:\files>script.bat  
show me as one line  
show  
me  
line  
by  
line
```

- בעזרת הסוגריים ניתן גם להריץ מספר פקודות בתוך הלולאה, כאשר ירידת שורה מפרידה ביניהם

```
for %%s in (show me line by line twice) do (  
    echo %%s  
    echo %%s  
)
```

```
C:\files>script.bat  
show  
show  
me  
me  
line  
line  
by  
by  
line  
line  
twice  
twice
```

לולאות

- ניתן להגדיר כאוסף ערכים את הקבצים בתיקייה, על ידי ביטוי תבנית
- ביטוי תבנית הוא כמו שם קובץ רגיל, רק שניתן להשתמש בסימן * כדי לייצג כל דבר

```
for %%s in (*) do echo %%s
```

```
C:\files>script.bat
input1.txt
input2.txt
input3.txt
script.bat
some_other_input.txt
```

```
for %%s in (*.txt) do echo %%s
```

```
C:\files>script.bat
input1.txt
input2.txt
input3.txt
some_other_input.txt
```

```
for %%s in (s*) do echo %%s
```

```
C:\files>script.bat
script.bat
some_other_input.txt
```

- לדוגמא: הצגת כל הקבצים שנמצאים בתיקייה

- ניתן גם להשתמש בתבנית כדי לסנן את הקבצים. דוגמאות:

- רק קבצי txt

- רק קבצים שמתחילים בs

- רק קבצי txt שמתחילים בs

```
for %%s in (s*.txt) do echo %%s
```

```
C:\files>script.bat
some_other_input.txt
```

לולאות

- דוגמא: סקריפט אשר עבור כל קובץ טקסט בתיקייה מדפיס את שמו ואת התוכן שלו

```
@echo off
for %%s in (*.txt) do (
    echo ==in file "%%s"==
    type %%s
    echo.
)
echo on
```

ירידת שורה

```
C:\files>script.bat
==in file "input1.txt"==
text in file: "input1.txt"
==in file "input2.txt"==
text in file: "input2.txt"
==in file "input3.txt"==
text in file: "input3.txt"
==in file "some_other_input.txt"==
1st line in file: "some_other_input.txt"
2nd line in file: "some_other_input.txt"
3rd line in file: "some_other_input.txt"
```

לולאות

- דוגמא: סקריפט אשר קולט מהמשתמש התחלה של שם קובץ, ומציג את כל קבצי הtxt שמתחילים בה

```
@echo off
set /p start=what should files start with?
for %%s in (%start%*.txt) do (
    echo %%s
)
echo on
```

```
C:\files>script.bat
what should files start with? s
some_other_input.txt
C:\files>script.bat
what should files start with? i
input1.txt
input2.txt
input3.txt
```

לולאות

- ניתן להוסיף את האפשרות /r כדי לעבור על הקבצים באופן רקרוסיבי – כלומר גם על תת-תיקיות

```
for /r %%f in (*.txt) do (  
    echo %%f  
)
```

```
C:\files>script.bat  
C:\files\input1.txt  
C:\files\input2.txt  
C:\files\input3.txt  
C:\files\some_other_input.txt  
C:\files\dir\inputInDir.txt  
C:\files\New folder\list_of_files.txt  
C:\files\New folder\new text file.txt
```

```

::@echo off
set /a x=1
for /l %%p in (1,1,5) do (
    echo %x%
    set /a x=%x%*2
)

```

לולאות

- שימו לב שבניגוד למשתנה הלולאה, משתנים רגילים נקראים בשלב הקריאה של הקובץ ולא בזמן הלולאה

- לדוגמא, נניח שאנו מנסים לכתוב סקריפט שמציג את 5 החזקות הראשונות של 2

- ניסון ראשון:

```

set /a x=1
for /l %%p in (1,1,5) do (
    echo %x%
    set /a x=%x%*2
)

```

```

C:\files>script.bat
1
1
1
1
1

```

- מה קרה?

- נבטל את echo off

```

C:\files>script
C:\files>set /a x=1
C:\files>for /L %p in (1 1 5) do (
echo 1
set /a x=1*2
)
C:\files>(
echo 1
set /a x=1*2
)
1
C:\files>

```

בהתחלה היה בא
1, לכן הפקודות
תורגמו ל
echo 1

לולאות

- הפקודה `setlocal enabledelayedexpansion`
- מאפשרת לנו להגדיר קריאה של משתנה רק בשלב ההרצה של הפקודה
- כאשר הפעלנו את הפקודה, אנו יכולים לגשת למשתנה עם ! במקום % כדי שהוא יחושב כך
- ניסיון שני

```
set /a x=1
setlocal enabledelayedexpansion
for /l %%p in (1,1,5) do (
    echo !x!
    set /a x=!x!*2
)
```

```
C:\files>script
1
2
4
8
16
```

- בנוסף בקריאה למשתנים ב! (לאחר שהופעלה הפקודה) אנו יכולים להרכיב שמות משתנים ביחס למשתנים אחרים, (נראה לזה דוגמא בהמשך)

לולאות

- ניתן להוסיף את האפשרות /L כדי להריץ לולאה על סדרת מספרים

(<start>, <step>, <end>)

- במקרה זה ה<set> בנוי בפורמט הבא

- <start> - הערך הראשוני

- <step> - בכמה מתקדמים

- <end> - הערך האחרון (תעצור כשתעבור אותו)

- לולאה עם משתנה %%i שקולה ללולאה

```
for(int i = <start>; i <= <end>; i += <step>)
```

- לדוגמא לולאה המציגה את כל המספרים האי-זוגיים בין 1 ל 10

```
for /L %%i in (1,2,10) do (  
    echo %%i  
)
```

```
C:\files>script.bat  
1  
3  
5  
7  
9
```

לולאות

- ניתן להוסיף את האפשרות /f כדי להריץ לולאה על שורות של קובץ

(<filename>)

- במקרה זה ה<set> בנוי בפורמט הבא

- <filename> - שם (או כתובת) הקובץ

- בנוסף יש להוסיף "tokens=*" לפני שם המשתנה (אחרת הוא ייקרא רק עד הרווח הראשון בכל שורה)

- לדוגמא:

```
@echo off
for /f "tokens=*" %%1 in (some_other_input.txt) do (
    echo %%1
)
echo on
```

```
C:\files>script.bat
1st line in file: "some_other_input.txt"
2nd line in file: "some_other_input.txt"
3rd line in file: "some_other_input.txt"
```

מערכים

- בעוד שבאופן עקרוני אין מערכים, ניתן לדמות התנהגות של מערכים ע"י שתילת מספרים בשמות משתנים כך שזה "נראה" כמו גישה לאינדקס במערך

```
set arr[0]=10
set arr[1]=55
set arr[2]=77
echo arr[0]=%arr[0]%
echo arr[1]=%arr[1]%
echo arr[2]=%arr[2]%
```

```
C:\files>script.bat
arr[0]=10
arr[1]=55
arr[2]=77
```

- לדוגמא: ה"מערך" [10,55,77]

- כדי לגשת ל"מערך" ע"י אינדקס שנמצא במשתנה נצטרך להרכיב את שם המשתנה בעזרת משתנה אחר – משתנה האינדקס

```
set arr[0]=10
set arr[1]=55
set arr[2]=77
setlocal enabledelayedexpansion
for /l %%i in (0,1,2) do (
    echo arr[%%i]=!arr[%%i]!
)
```

```
C:\files>script.bat
arr[0]=10
arr[1]=55
arr[2]=77
```

- ניתן לעשות זאת במשתנים הנקראים עם !

תרגול

- כתבו סקריפט אשר:
- קולט מהמשתמש n – אורך "מערך"
- קולט n מספרים ושומר אותם ב"מערך"
- קולט מהמשתמש מספר שבו צריך להכפיל את כל הערכים
- מכפיל את כל ערכי ה"מערך" בערך שהתקבל
- מדפיס את ערכי ה"מערך" המעודכנים
- כל שלב צריך להתבצע בנפרד
- דוגמא

```
C:\files>array_ex.bat  
array size=3  
arr[0]=5  
arr[1]=66  
arr[2]=700  
mult by? 3  
arr[0]=15  
arr[1]=198  
arr[2]=2100
```

תרגול

```
@echo off
set /p n=array size=
set /a lastI=n-1
for /l %%i in (0,1,%lastI%) do (
    set /p arr[%%i]=arr[%%i]=
)
set /p mult=mult by?
setlocal enabledelayedexpansion
for /l %%i in (0,1,%lastI%) do (
    set /a arr[%%i]=%mult%*!arr[%%i]!
)
for /l %%i in (0,1,%lastI%) do (
    echo arr[%%i]=!arr[%%i]!
)
echo on
```

- כתבו סקריפט אשר:

- קולט מהמשתמש n – אורך "מערך"

- קולט n מספרים ושומר אותם ב"מערך"

- קולט מהמשתמש מספר שבו צריך להכפיל את כל הערכים

- מכפיל את כל ערכי ה"מערך" בערך שהתקבל

- מדפיס את ערכי ה"מערך" המעודכנים

- כל שלב צריך להתבצע בנפרד

- דוגמא

```
C:\files>array_ex.bat
array size=3
arr[0]=5
arr[1]=66
arr[2]=700
mult by? 3
arr[0]=15
arr[1]=198
arr[2]=2100
```

ארגומנטים

```
@echo off  
echo %1  
echo %2  
echo on
```

```
C:\files>script hello world  
hello  
world  
C:\files>script thank you  
thank  
you
```

- כמו תוכנה, ניתן להפעיל סקריפט עם ארגומנטים
- כדי לגשת לארגומנט הראשון ניגש למשתנה %1
- כדי לגשת לארגומנט השני ניגש למשתנה %2
- וכך הלאה
- בלולאה, הביטוי (%*) מייצר אוסף של כל הארגומנטים

```
@echo off  
for %%a in (*) do echo %%a  
echo on
```

```
C:\files>script two args  
two  
args  
C:\files>script now three args  
now  
three  
args
```

תרגול

- כתבו סקריפט אשר מקבל כארגומנט (כלומר, ניתן בהפעלה) כמות מספרים
- לאחר מכן ייקלטו מהמשתמש כמות המספרים שהוגדרה, ויוצג מה הערך המקסימלי שהוזן
- דגומאות

```
C:\files>ex 3
#1: 5
#2: 12
#3: 10
max is 12
C:\files>ex 5
#1: 1
#2: -5
#3: 15
#4: 7
#5: -9
max is 15
```

תרגול

- כתבו סקריפט אשר מקבל כארגומנט (כלומר, ניתן בהפעלה) כמות מספרים
- לאחר מכן ייקלטו מהמשתמש כמות המספרים שהוגדרה, ויוצג מה הערך המקסימלי שהוזן
- דגומאות

```
@echo off
set /a max=0
setlocal enabledelayedexpansion
for /l %%i in (1,1,10) do (
    set /p x=#%%i:
    if !x! GTR !max! (set max=!x!)
)
echo max is %max%
echo on
```

```
C:\files>ex 3
#1: 5
#2: 12
#3: 10
max is 12
C:\files>ex 5
#1: 1
#2: -5
#3: 15
#4: 7
#5: -9
max is 15
```